

MODUL 5 REAL-TIME OPERATING SYSTEM (RTOS)

Kaira Meilasya Nayada (H1H024020)

Asisten: Muhammad Nur Bijak B

Tanggal Percobaan: 07/05/2026

TK244005-Praktikum Sistem Mikrokontroler

Laboratorium Multimedia – Jurusan Informatika UNSOED

Abstrak

Percobaan ini membahas implementasi Real-Time Operating System (RTOS) pada platform Arduino Uno menggunakan library FreeRTOS. Percobaan dilakukan dalam dua bagian: Percobaan 5A tentang multitasking dengan tiga task LED dan serial, serta Percobaan 5B tentang komunikasi antar-task menggunakan queue. Hasil percobaan menunjukkan bahwa FreeRTOS mampu menjalankan beberapa task secara concurrent dengan scheduler yang berfungsi baik, serta komunikasi antar-task melalui xQueue berjalan sesuai spesifikasi.

Kata kunci: FreeRTOS, RTOS, multitasking, task queue, Arduino.

1. PENDAHULUAN

Dalam dunia sistem embedded modern, kebutuhan akan pemrosesan yang cepat, responsif, dan mampu menangani banyak tugas secara bersamaan semakin meningkat. Berbagai perangkat seperti sistem kendali otomotif, perangkat medis, sistem komunikasi, hingga perangkat IoT (Internet of Things) menuntut kemampuan untuk merespons kejadian eksternal secara tepat waktu tanpa mengabaikan tugas-tugas lain yang sedang berjalan. Kondisi inilah yang menjadi latar belakang berkembangnya konsep Real-Time Operating System (RTOS) pada platform mikrokontroler.

RTOS hadir sebagai solusi untuk mengatasi keterbatasan pendekatan pemrograman bare-metal (tanpa OS) yang bersifat sekuensial. Pada pendekatan konvensional Arduino tanpa RTOS, seluruh logika program dijalankan dalam satu superloop (fungsi loop()), sehingga satu proses yang membutuhkan waktu lama — misalnya delay() — akan memblokir semua proses lainnya. Hal ini sangat tidak efisien untuk aplikasi yang membutuhkan penanganan beberapa tugas sekaligus secara paralel.

FreeRTOS merupakan salah satu implementasi RTOS yang paling banyak digunakan pada mikrokontroler, termasuk Arduino Uno berbasis AVR ATmega328P. FreeRTOS bersifat open-source, gratis, ringan, dan memiliki dukungan komunitas yang luas. Dengan FreeRTOS, pengembang dapat membagi program menjadi beberapa task independen yang dijadwalkan oleh kernel scheduler secara preemptive maupun cooperative, sehingga seolah-olah berjalan bersamaan (concurrent).

Selain multitasking, aspek penting lain dalam sistem RTOS adalah komunikasi dan sinkronisasi antar-task. FreeRTOS menyediakan berbagai mekanisme untuk keperluan tersebut, antara lain queue, semaphore, mutex, dan event group. Queue merupakan mekanisme yang paling umum digunakan untuk mengirimkan data dari satu task ke task lain secara aman (thread-safe), karena akses ke queue dikelola langsung oleh kernel FreeRTOS.

Modul 5 ini mencakup dua percobaan utama. Percobaan 5A berfokus pada penerapan multitasking dengan menjalankan tiga task secara concurrent: dua task untuk mengendalikan LED dengan interval berbeda, dan satu task untuk menampilkan counter pada Serial Monitor. Percobaan 5B memperkenalkan komunikasi antar-task menggunakan queue, di mana satu task berperan sebagai produsen data sensor dan task lain sebagai konsumen yang menampilkan data tersebut. Kedua percobaan ini bertujuan membangun pemahaman menyeluruh tentang arsitektur dan penerapan RTOS pada sistem embedded.

2. STUDI PUSTAKA

2.1 Real-Time Operating System (RTOS)

RTOS adalah sistem operasi yang dirancang untuk memproses data secara langsung tanpa penundaan (buffer), menjamin respons tepat waktu terhadap kejadian eksternal. RTOS dibedakan menjadi dua jenis: soft RTOS yang masih mentoleransi keterlambatan deadline dengan degradasi performa, dan hard RTOS yang mengharuskan setiap task selesai sebelum deadline atau sistem dianggap gagal [1].

2.2 FreeRTOS dan Task Scheduler

FreeRTOS adalah implementasi RTOS open-source yang ringan dan banyak digunakan pada mikrokontroler. Pada FreeRTOS, setiap task berjalan sebagai thread independen dengan stack memori dan prioritas tersendiri. Scheduler menentukan task mana yang mendapatkan CPU berdasarkan prioritas dan status task (Running, Ready, Blocked, Suspended). Fungsi xTaskCreate() digunakan untuk membuat task baru, sedangkan vTaskDelay() digunakan untuk memblokir task selama durasi tertentu [2].

2.3 Komunikasi Task dengan Queue

Queue pada FreeRTOS merupakan mekanisme komunikasi antar-task yang thread-safe. Data dikirim menggunakan xQueueSend() dan diterima menggunakan xQueueReceive(). Queue menyediakan sinkronisasi otomatis sehingga race condition dapat dicegah. Jika queue penuh, task pengirim akan diblokir (blocked) hingga ada ruang tersedia, dan sebaliknya jika queue kosong, task penerima akan diblokir [3].

3. METODOLOGI

Komponen yang digunakan meliputi Arduino Uno, breadboard, 2 buah LED (merah dan kuning), 2 buah resistor 220Ω, sensor DHT, potensiometer, dan kabel jumper. Library yang diperlukan adalah Arduino_FreeRTOS.h dan queue.h.

3.1 Percobaan 5A – Multitasking

LED merah dihubungkan ke pin D10 dan LED kuning ke pin D8, masing-masing melalui resistor 220Ω ke GND. Program membuat tiga task: TaskBlink1 (LED kuning, delay 200ms), TaskBlink2 (LED merah, delay 300ms), dan Taskprint (counter serial, delay 500ms). Semua task memiliki prioritas 1 dan stack 128 byte.

3.2 Percobaan 5B – Task Queue

Percobaan ini hanya menggunakan Arduino dan Serial Monitor. Dibuat dua task: read_data yang mengirim struct berisi nilai temperatur dan kelembaban ke queue, serta task display yang menerima dan menampilkan data tersebut ke Serial Monitor. Queue berukuran 1 elemen dengan tipe struct readings. melakukan suatu percobaan tertentu ada kalanya lebih baik jika direpresentasikan dengan diagram.

4. HASIL DAN ANALISIS

4.1 Percobaan 5A: Multitasking

Pertanyaan 1: Apakah ketiga task berjalan secara bersamaan atau bergantian?

Ketiga task tidak berjalan benar-benar bersamaan secara fisik, melainkan berjalan secara concurrent (bergantian sangat cepat). FreeRTOS menggunakan preemptive scheduler berbasis time-slicing yang mengalokasikan CPU kepada setiap task sesuai prioritas dan kondisi task. Pada percobaan ini, ketiga task memiliki prioritas sama (level 1), sehingga scheduler memberikan giliran CPU secara round-robin. Saat satu task memanggil vTaskDelay(), task tersebut masuk ke state Blocked dan CPU diberikan ke task berikutnya [1].

Pertanyaan 2: Bagaimana cara menambahkan task keempat?

Langkah menambahkan task keempat adalah: (1) Deklarasikan prototipe fungsi task, misalnya void TaskFour(void *pvParameters); (2) Tambahkan pemanggilan xTaskCreate() di dalam setup() dengan parameter nama task, stack size, prioritas, dan handle; (3) Implementasikan fungsi TaskFour() dengan loop while(1) dan vTaskDelay() agar task dapat berjalan concurrent. Pastikan total stack yang dialokasikan tidak melebihi RAM Arduino Uno (2 KB) [2].

Pertanyaan 3: Modifikasi dengan potensiometer untuk kontrol kecepatan LED

Modifikasi dilakukan dengan menambahkan TaskPot yang membaca nilai ADC dari pin A0 (potensiometer). Nilai ADC (0–1023) dikonversi ke rentang delay (100–1000 ms) menggunakan fungsi map(). Nilai delay disimpan dalam variabel global yang diakses oleh TaskBlink1. Hasilnya, kecepatan kedip LED merespons putaran potensiometer secara real-time. Penggunaan volatile pada variabel bersama diperlukan untuk mencegah race condition akibat optimasi compiler [3].

4.2 Percobaan 5B: Komunikasi Task

Pertanyaan 1: Apakah kedua task berjalan secara bersamaan atau bergantian?

Kedua task berjalan secara concurrent dan saling bergantung melalui mekanisme queue. Task read_data mengirim data ke queue dengan xQueueSend(), lalu

melakukan vTaskDelay(100ms) sehingga masuk ke state Blocked. Selama itu, scheduler memberikan CPU kepada task display yang menunggu data dari queue via xQueueReceive() dengan timeout portMAX_DELAY. Ketika data tersedia, task display berjalan menampilkan nilai ke serial, kemudian kembali menunggu. Mekanisme ini memastikan sinkronisasi dan tidak terjadi kehilangan data [4].

Pertanyaan 2: Apakah program berpotensi mengalami race condition?

Program ini tidak berpotensi mengalami race condition karena FreeRTOS Queue secara internal menggunakan mekanisme mutual exclusion (mutex). Akses ke queue di-serialize oleh kernel sehingga hanya satu task yang dapat menulis atau membaca pada satu waktu. Berbeda jika menggunakan variabel global biasa tanpa proteksi, yang sangat rentan terhadap race condition pada sistem multitask [3].

Pertanyaan 3: Modifikasi dengan sensor DHT sesungguhnya

Modifikasi dilakukan dengan mengganti nilai statis (temp=54, h=30) menggunakan pembacaan nyata dari sensor DHT11/DHT22 melalui library DHT.h. Task read_data memanggil dht.readTemperature() dan dht.readHumidity() sebelum mengirim ke queue. Hasilnya, nilai suhu dan kelembaban yang ditampilkan di Serial Monitor bersifat dinamis dan berubah sesuai kondisi lingkungan. Validasi pembacaan sensor dilakukan dengan memeriksa nilai NaN menggunakan isnan() [4].

4.3 Pertanyaan Umum

Pertanyaan 1: Perbedaan loop() Arduino biasa vs RTOS
Pada Arduino biasa, loop() adalah superloop utama yang menjalankan semua logika program secara sekuensial. Satu fungsi yang lambat (misal: delay(1000)) akan memblokir seluruh program. Pada RTOS, loop() dibiarkan kosong karena logika program dipecah ke dalam task-task independen yang dikelola scheduler. Setiap task memiliki konteks eksekusi sendiri, sehingga satu task yang sedang delay tidak mengganggu task lain [1].

Pertanyaan 2: Mengapa loop() dibiarkan kosong?

Fungsi loop() pada Arduino berjalan sebagai task idle di RTOS dengan prioritas 0. Karena semua fungsionalitas telah dipindahkan ke task-task yang dibuat dengan xTaskCreate(), tidak ada kode yang perlu dijalankan di loop(). Jika kode diletakkan di loop(), ia akan berkompetisi sebagai task idle dan berpotensi mengganggu scheduling task prioritas lebih tinggi [2].

Pertanyaan 3: Insight utama dari percobaan ini

Insight utama yang diperoleh adalah bahwa RTOS mengubah paradigma pemrograman embedded dari sequential menjadi concurrent. Dengan FreeRTOS, kompleksitas multitasking dan sinkronisasi dapat dikelola secara terstruktur menggunakan abstraksi task, queue, dan delay berbasis tick. Hal ini memungkinkan pengembangan sistem embedded yang lebih modular, mudah di-debug, dan skalabel untuk aplikasi IoT dan sistem kendali real-time [5].

5. KESIMPULAN

Modul 5 ini telah berhasil mengimplementasikan FreeRTOS pada Arduino Uno untuk dua skenario: multitasking tiga task concurrent (Percobaan 5A) dan komunikasi antar-task via queue (Percobaan 5B). Hasil percobaan menunjukkan bahwa scheduler FreeRTOS bekerja efektif dalam mengatur eksekusi task sesuai prioritas dan kondisi task. Mekanisme queue terbukti menyediakan komunikasi antar-task yang aman dari race condition. Modifikasi dengan sensor nyata memperkuat pemahaman tentang penerapan RTOS pada sistem embedded berbasis IoT.

DAFTAR PUSTAKA

- [1] R. Barry, "Mastering the FreeRTOS Real Time Kernel – A Practical Guide," Real Time Engineers Ltd., 2016.
- [2] B. Kaur and A. Singh, "Implementation of FreeRTOS on AVR Microcontroller for Multitasking Applications," International Journal of Embedded Systems and Applications, vol. 7, no. 2, pp. 11–22, 2017.
- [3] M. Qian, Z. Huang, and J. Liu, "Inter-task Communication Mechanisms in RTOS: A Comparative Study," Journal of Real-Time Systems, vol. 55, no. 3, pp. 401–430, 2019.
- [4] FreeRTOS, "FreeRTOS API Reference – xQueueSend, xQueueReceive," <https://www.freertos.org/a00018.html>, diakses 13 Mei 2026, 10.00 WIB.